Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	Ctrl + Future
Project / Solution Name	SME Smart
Name (Leader)	Teh Sze Yu Jocelyn
Email & Phone (Leader)	Jocelyteh0602@gmail.com / 012-9202655
University Affiliation(s)	Universiti Teknologi Malaysia (UTM) / Universiti Putra Malaysia (UPM)
Live Prototype URL	https://sme-smart.replit.app/
Code Repository URL (public / access granted to judges)	https://github.com/gohyiqi0621/SDC-Hackathon/

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

SME Smart is a web portal for small businesses. Users open the app in a browser at sme-smart.replit.app. The app is built with PHP 8.2 and runs on Replit Deployments (Autoscale) behind Nginx and PHP-FPM. All data, including users, products, orders, invoices, claims, login sessions and uploaded files, is stored in a PostgreSQL 16 database on Replit.

Customers chat with the Customer Assistant on Telegram (and a website test chat).Telegram messages reach the app through a webhook. The app forwards each conversation to Flowise, hosted on Railway, which runs the AI workflow using Claude Haiku 4.5 or Agnes 2.5 Flash. Flowise calls our own API endpoints as tools to check products, place orders and book appointments.

Staff use the Staff Assistant and the dashboard. Gmail and Google Calendar are connected through Google OAuth 2.0. Email summaries, reply drafts and SOPs are generated by a Qwen2.5-7B model hosted on RunPod Serverless, with Claude and Agnes as fallbacks. Expense-claim receipts are read by the model the business selects (Claude Haiku 4.5 or Agnes 2.5 Flash), with Claude Haiku 5.5 as a fallback.Invoices and receipts are generated as PDF files inside the app.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
PHP 8.2	Main backend: authentication, business logic, database access, invoices and API integration.	PHP License
Nginx + PHP-FPM (Replit deployment)	Web server and PHP process manager that serve the app on Replit.	Nginx: BSD-2-Clause; PHP-FPM: PHP License
HTML5, CSS, JavaScript	Website structure, styling, dashboard interactions and browser functionality.	Language standards; no separate library license

Name & Version	Purpose in Project	License
PostgreSQL — PostgreSQL 16 configured in Replit	Stores users, products, orders, invoices, claims, bookings, and integration settings.	PostgreSQL License
Python 3.13 (Replit environment)	RunPod serverless handler and the hackathon template scripts.	Python Software Foundation License
Custom PHP MVC structure	Project's own routing, controllers, models and views.	Project repository declares Apache-2.0
Three.js 0.152.2, including GLTFLoader	Displays and animates the dashboard's 3D robot.	MIT License
Font Awesome Free 6.5.2	Interface icons	Code: MIT; fonts: SIL OFL 1,1; icons:CC BY 4.0. License
PDF.js 3.11.174	Renders uploaded PDF pages for document processing.	Apache 2.0
tFPDF 1.34	Generates invoice and official receipt PDFs with Unicode fonts.	LGPL-2.1, according to the bundled README
Inter font	Website typography, loaded through Google Fonts.	SIL Open Font License
DejaVu Sans fonts	Fonts for PDF generation.	Bundled Bitstream Vera/DejaVu font terms
Flowise 3.1.2	Orchestrates customer/staff assistants and tool calls.	Apache 2.0 for core; some components have separate commercial terms. License

Name & Version	Purpose in Project	License
RunPod Python SDK — unpinned, optional	Serverless endpoint handler	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	**Purpose in Project**	**Access Tier (free / paid / sponsor-provided)**
Flowise	Chatbot engine for Customer Assistant and Staff Assistant.	Free
Anthropic (Claude)	AI auto-fill for Claims (receipt reading) and serves as one of the chatbot models.	Paid
Runpod	AI for text generation, such as generating SOPs.	Sponsor-provided
Agnes AI	AI for generating SOPs and acts as a backup provider.	Free
Telegram Bot API	Used for customer chats and orders via Telegram. Also sends staff notifications, QR payments, and official receipts.	Free
Brevo (transactional email API)	Sends system emails such as password-reset links.	Free
Google OAuth 2.0	Connects user Google accounts	Free
Gmail API	Allows the Staff Assistant to read emails, categorize them, and send emails.	Free
Google Calendar API	Enables the Customer Assistant to create booking appointments.	Free

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit Deployments (Replit)	Hosts the SME Smart web application (PHP 8.2, Nginx, PHP-FPM) at sme-smart.replit.app. Replit Agent was also used to build, debug and publish the app.	Replit Core ($20/month), Autoscale deployment
Replit PostgreSQL (Replit)	Main database for users, products, orders, invoices, receipts, claims and chat history. Also stores login sessions and backups of uploaded files, so data survives restarts and redeploys.	PostgreSQL 16, built into Replit Core
RunPod Serverless (RunPod)	GPU compute for the Qwen2.5-7B-Instruct model (served with vLLM), used for text generation: assistant SOPs, email summaries and reply drafts.	Serverless, pay-as-you-go; 24 GB and 80 GB GPU workers
Flowise on Railway (Railway)	Hosts the Flowise AI workflows that power the Customer Assistant and Staff Assistant chatbots.	Railway cloud hosting
Google Cloud OAuth 2.0 (Google)	Authentication for connecting Gmail and Google Calendar, so the Staff Assistant can read and send emails and the Customer Assistant can book appointments.	Google Cloud project with OAuth consent screen
Telegram Bot API (Telegram)	Customer chat channel and staff notifications, delivered to the app through a webhook.	Free
Brevo (Brevo)	Sends system emails such as password reset links.	Free plan

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Qwen2.5-7B -Instruct (served with vLLM)	Alibaba Qwen (open weights), self-hosted on RunPod Serverless	Pre-trained. Primary text model: summarises emails, drafts email replies and writes assistant SOPs.	Apache License 2.0
Claude Haiku 4.5 (claude-haiku-4-5)	Anthropic API, via Flowise	Pre-trained. Default model for the Customer Assistant and Staff Assistant chatbots; reads expense-claim receipts; fallback for text generation.	Proprietary; Anthropic Commercial Terms of Service
Claude Haiku 5.5 (claude-haiku-5-5)	Anthropic API	Pre-trained. Fallback model for reading uploaded receipts and auto-filling expense claims.	Proprietary; Anthropic Commercial Terms of Service
Agnes 2.5 Flash (agnes-2.5 Flash)	Agnes AI API	Pre-trained. Alternative chatbot model, can read expense-claim receipts, and final fallback for text generation.	Proprietary; Agnes AI API Terms of Service

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Business product & service catalogue	Entered or imported (CSV/Excel) by each business in the app	Private, self-collected	Owned by each business	**N**
Company profile & assistant SOPs	Entered by each business; SOPs drafted by AI and reviewed by the business	Private, self-collected	Owned by each business	**N** (business contact details only)
Customer chat, order & booking records	Collected through the Customer Assistant on Telegram and web chat	Private, self-collected	Owned by each business; handled under our Privacy Policy and PDPA 2010	**Y** (names, Telegram usernames, phone numbers, messages, payment references)
Gmail and Google Calendar data	User's own Google account via Google APIs (OAuth), processed at run time	Private, user-authorised	Google API Services User Data Policy (Limited Use)	**Y** (email content, contacts, appointments)
Receipts, bank transfer slips & expense claims	Uploaded by staff and customers in the app	Private, self-collected	Owned by each business	**Y** (names, amounts, bank references)

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☒ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
OpenAI Codex	Code generation and edits, UI changes, debugging, feature planning and documentation.	Major
Replit Agent	Replit deployment setup (Nginx / PHP-FPM, Autoscale), environment configuration and debugging.	Moderate
Google Gemini	Generated the SME Smart logo and image assets.	Major
Claude (Claude Code / Claude app)	Code generation, debugging, invoice / receipt PDF and UI design, responsive layout fixes and documentation.	Major

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

We used AI tools to help us with coding, debugging, design ideas, and documentation. However, we did not rely entirely on AI-generated results. Our team reviewed the suggested code, tested different features, and made changes whenever something did not work as expected. We also checked the content to make sure it matched our project requirements. Each team member was responsible for reviewing their assigned tasks, and we worked together to fix issues before the final submission. AI helped speed up our development, but the final decisions and responsibility remained with our team.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☒ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Cloud development / hosting resources	Used to develop, test, and deploy the SME Smart web application.
RunPod	GPU computing resource	Used to run Qwen2.5-7B-Instruct with vLLM to generate customized SOPs, email summaries and reply drafts.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Replit and RunPod were important in helping us build SME Smart. We used Replit to develop, test, and deploy our web application, while RunPod helped us generate customized SOP prompts for our AI assistants. Both platforms made it easier for us to complete our prototype within the limited time. However, they are not the only options, as we could use other cloud hosting and GPU services in the future.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

SME Smart stores business data (company profile, products, orders, invoices, claims) and customer data from Telegram (name, username, phone number, messages, bank slips) in PostgreSQL on Replit. Messages are sent to Flowise and the selected AI provider (Anthropic or Agnes) only to generate replies; claim receipts are sent to the AI only to read their details. Gmail and Calendar data is read through Google OAuth when needed; only the connection tokens are stored. API keys and tokens are encrypted (AES-256-GCM), passwords are hashed and uploaded files are private.

How long data is kept: business and customer records stay until the business deletes them; login sessions end after 60 minutes of inactivity and are removed after 8 hours; password-reset links expire after 60 minutes; the troubleshooting log and the Staff Assistant's email-list cache are temporary server files, cleared on every redeploy. We follow PDPA 2010.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

AI models can misunderstand a request or invent details (hallucination). Mitigations: orders are created only after the customer confirms them; the app checks every order number the AI mentions against the database and blocks invented ones; bookings are checked against Google Calendar; payments are never approved by AI, as staff verify every bank slip.

Other risks: dependence on external providers (RunPod, Anthropic, Agnes, Flowise), reduced by automatic fallback from RunPod to Claude to Agnes; RunPod cold starts can delay SOP generation; Telegram and Google API rate limits; and our Google OAuth app is still awaiting verification, so only approved test accounts can connect Gmail and Calendar.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- [/] All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- [/] All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- [/] No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- [/] Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- [/] Any use of official SDC sponsor resources has been declared in Section 5.
- [/] The prototype described here is accessible to judges through the links provided in Section 1.
- [/] All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature] *Jocelyn*

[Date] 23/10/2026